

JMPS

Documento Técnico

VulnScanner

Análise Técnica da Arquitetura e Varredura de Segurança Web em Node.js

VulnScanner	1
1. Visão Geral do Arquivo e Arquitetura.....	1
2. Detalhamento dos Componentes Principais.....	2
A. Backend (server.js)	2
B. Módulos de Escaneamento (/scanners).....	2
C. Laboratório de Testes (/local-lab).....	3
4. Recomendações de Melhoria.....	4

[Manual De Uso](#)

Preparado por: João Martins Pinto Soares / Desenvolvedor

Data: 03/08/2026

Versão: 1.0

1. Visão Geral do Arquivo e Arquitetura

O projeto é uma **aplicação web em Node.js (Express)** para **varredura de vulnerabilidades de segurança web**. Ele opera em um modelo cliente-servidor leve, onde a interface envia requisições HTTP para uma API Node.js que orquestra módulos de escaneamento passivo e ativo.

```
VulnScanner-main/
├── README.md & funcionamento.txt # Documentação de operação e escopo
├── vuln-scanner-web/
│   ├── server.js # Aplicação Principal Node.js
│   ├── package.json # Ponto de entrada (API REST com Express)
│   ├── public/ # Dependências (Express, Cheerio, Undici)
│   │   ├── index.html # Interface web do usuário (Frontend)
│   │   ├── app.js
│   │   └── style.css
│   └── scanners/ # Motores de análise técnica
│       ├── passiveScanner.js # Varredura passiva de cabeçalhos e configurações
│       ├── activeScanner.js # Varredura ativa (fuzzing básico/XSS/SQLi)
│       ├── exploiter.js # Módulo de verificação/PoC básica de exploração
│       └── references.js # Banco de dados de referências/remendos
└── local-lab/ # Laboratório Docker local para testes
    └── docker-compose.yml
```

2. Detalhamento dos Componentes Principais

A. Backend (server.js)

- **Framework:** Express.js rodando no Node.js.
- **Rotas Principais:**
 - `/api/scan`: Recebe a URL alvo e os parâmetros de escaneamento selecionados na interface (passivo, ativo ou validação).
 - Orquestra a chamada assíncrona dos módulos dentro do diretório `/scanners`.

B. Módulos de Escaneamento (/scanners)

1. **Scanner Passivo (passiveScanner.js):**
 - a. Analisa os cabeçalhos de resposta HTTP sem alterar o comportamento do servidor alvo.
 - b. Identifica a ausência de cabeçalhos de segurança essenciais: Content-Security-Policy (CSP), X-Frame-Options, X-Content-Type-Options, Strict-Transport-Security (HSTS).
 - c. Verifica a exposição de versões de servidor/tecnologias nos cabeçalhos Server e X-Powered-By.
2. **Scanner Ativo (activeScanner.js):**
 - a. Realiza requisições manipuladas para descobrir pontos de injeção em formulários, parâmetros URL ou endpoints.
 - b. Testa vetores simples como **XSS Refletido** (injeção de payload em parâmetros) e **SQL Injection** (caracteres de controle como ' e ", analisando respostas de erro do banco de dados).
3. **Exploiter (exploiter.js):**
 - a. Módulo auxiliar para validação ou "Proof of Concept" (PoC) de vulnerabilidades encontradas, simulando requisições com carga útil para confirmar falhas sem execução nociva.
4. **Base de Referências (references.js):**
 - a. Mapeia as vulnerabilidades encontradas para identificadores padrão (como referências CWE/OWASP) e sugere recomendações de correção.

C. Laboratório de Testes (/local-lab)

- Inclui um arquivo docker-compose.yml para levantar rapidamente um ambiente vulnerável controlado. Isso facilita o teste local das regras de escaneamento sem afetar sistemas de terceiros.

Categoria	Ponto Forte	Ponto Crítico / Risco
Arquitetura	Código bem modularizado em responsabilidades isoladas (passivo vs. ativo).	Falta de tratamento robusto para limites de taxa (<i>Rate Limiting</i>) no servidor Express.
Requisições HTTP	Utiliza bibliotecas modernas (undici/cheerio para parsing).	Risco de SSRF (Server-Side Request Forgery) se o backend escanear URLs internas (ex: 127.0.0.1 ou 169.254.169.254) a pedido do usuário.
Análise Ativa	Boa abordagem inicial para detectar falhas comuns OWASP Top 10.	Vetores de testes baseados em assinaturas/payloads simples (pode gerar falsos positivos/negativos).

4. Recomendações de Melhoria

- Proteção contra SSRF:** Validar e sanitizar rigorosamente a URL de destino recebida no `server.js`, bloqueando endereços IP privados (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 127.0.0.1).
- Arquivos da Node Modules no Repositório:** A pasta `node_modules/` foi incluída no arquivo comprimido (.zip). É recomendado adicioná-la ao arquivo `.gitignore` para manter o repositório enxuto e gerenciar dependências exclusivamente pelo `package.json` (`npm install`).
- Filas e Assincronismo:** Para escaneamentos mais longos, considerar o uso de filas (como *Bull* ou *RabbitMQ*) em vez de tratar requisições diretamente de forma síncrona/bloqueante no Express.